

INF-2200

Introduction to Assembly

N.M. Øyvind G.A. Ragnhild O.K. Jørgen T.N. Vi Lars Ailo Bongo

University of Tromsø

*oyvind.a.nohr@uit.no ragnhild.a.grape@uit.no jorgen.k.olsen@uit.no
vi.n.tran@uit.no lars.ailo.bongo@uit.no*

Computer Architecture
May 29, 2026

Overview I

Instruction set architecture (ISA)

ISA

- An ISA enables multiple different hardware to implement the same functionality in different ways. That is why you can play your games on both AMD and Intel processors
- We will focus on the **x86-64** ISA, but there exist many more and perhaps most used is the **ARM**-family
- As we know from previous courses, memory can be viewed as a collection of bits
- This ISA can only access **groups** of bits at a time
- 1, 2, 4 or 8 bytes (1 byte==8bits)

- A 64-bit architecture means we can access a 64 bit address-space or 2^{64} addresses
- 2^{64} gives us the address range: [0x0000000000000000 - 0xFFFFFFFFFFFFFFFF] (18EB)
- In practice only 52/57 bit address space used (AMD/Intel)
- As of 2023 no commercial motherboard support this amount of physical memory

- As of 2023, largest commodity dimm chip is samsung's 512GB so-dimm, and 18EB would require **35156250** chips
- Given a so-dimm form factor of $3 \times 6.92\text{cm}$ 18EB memory would take up 72984.375m^2 or about seven football-pitches with samsung's chips
- In 2017, HPE released a 160TB single compute system
- If Moore's law continues we can expect a doubling of density every two years, and reach 18EB roof in about 30 years. (What is the prediction for Moore's law?)

Benefits of assembly

Assembly

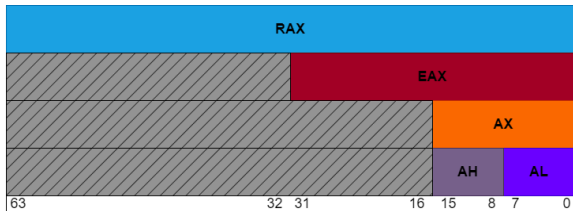
- On a low level, programming is all about moving numbers around; Assembly is in many ways a human readable binary
 - The direct mapping from assembly to binary is complex
 - E.g. assembly for 8080 can be assembled with an 8086 assembler to produce a working 8086 binary even though 8080 and 8086 is not binary compatible
- Assembly does not have variables in the traditional sense
- We instead use temporary, quick storage units called *registers*

Registers

- x86-64 gives us 16 GPRs (General purpose registers)
- These are 64bit *integer* registers
- We can access individual lower halves of the registers, see <https://learn.microsoft.com/en-us/windows-hardware/drivers/debugger/x64-architecture> for an overview of suffixes to access partials of registers

Registers

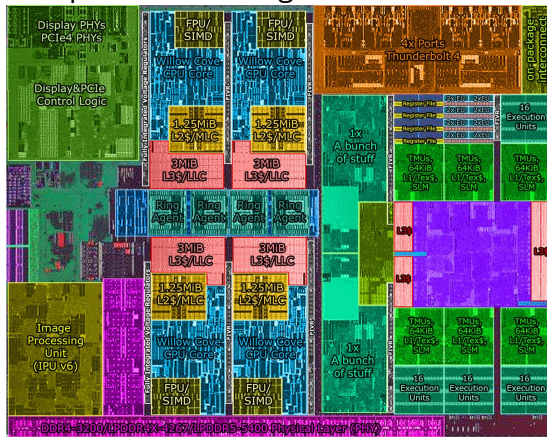
Assembly



Name	Legacy Usage
AX	Accumulator register
CX	Counter register
DX	Data register
BX	Base register
SP	Stack pointer
BP	Base pointer
SI	Source index register
DI	Destination index register
R8-R15	Numbered general registers

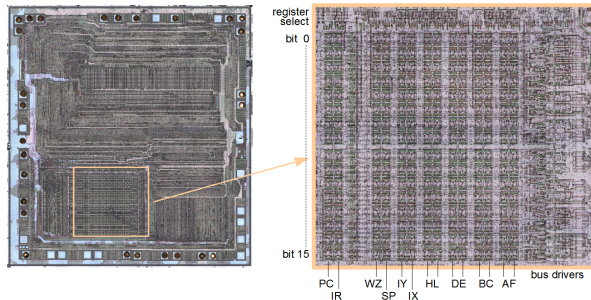
Die shots

Complete die intel tiger lake



source: Intel tiger lake

Register closeup Z80



source: Z80

Registers vs. memory

- For assignments 1 and 3 we will use *integer* registers
- Moving values between registers is extremely quick, a few clock-cycles
- Accessing *memory* takes **hundreds**
- In the x86-64 ISA we are not allowed to move numbers directly between memory locations
- This means, you will have to utilize those registers!

Assembly syntax

Assembly

- Two syntaxes are used for x86-64 assembly: *Intel* and *AT&T/GAS*
- We will use **AT&T**
- Be careful when looking up sources online, many will use the **Intel** syntax
- How do we know? AT&T will use **%** as a prefix for registers: %rax

ATT

```
1  movl  $0, %eax
```

Intel

```
3  mov  eax, 0
```

Instructions

Assembly

- There are many instructions in the x64(short for x86-64) ISA
- To begin with we will have a look at **INC**
- Inc increments the value of a target operand by '1'
- We can suffix the instruction with some letters to say what data size we are working with:
- *b*: byte(1B), *w*: word(2B), *l*: long(4B), *q*: quad(8B)

More instructions

Assembly

- *DEC* Decrease value
- *ADD* Add <src> to <dest>
- *SUB* Subtract <src> from <dest>
- Immediate values are prefixed with \$
- Comments start with #

```
1  add $2, %al
2  addw $2, %bx
3  addl $2, %ecx
4  addq $2, %rdx
```

```
1  subb $1, %r8b # subtract 1 from the least significant byte of r8
2  subw $1, %r9w # subtract 1 from the least significant word of r9
3  subl $1, %r10d # subtract 1 from the least significant double word of r10
4  subq $1, %r11 # subtract 1 from the r11 register
```

More Instructions

Assembly

- *MOV* move data; into a register, between two registers, or to memory
- **Remember** you cannot move data between two memory locations directly

```
1  mov $1, %rax # Move 1 into rax
2  mov %rax, %r8 # Move rax into r8
3  mov %rax, (%r8) # *(r8) = rax, move rax into the address pointed by r8
```

An important abstraction we know from previous courses is pointers. But in assembly they behave a bit different than in the C language. In **assembly** pointers:

- Does **not** increment by a multiple of the size of the data type it points to. This is perhaps more intuitive
 - ++ is **+1** not **+sizeof(type)**
- We access pointers (memory addresses) by having parenthesis around a register whose value is the memory address, see previous slide

Program flow

Assembly

- Instructions are read **sequentially**
- The x86-ISA provides guarantees that instructions are executed **in-order**, meaning you will see the effect of instruction **1** before instruction **2** is executed
- This is not completely true, but a sufficient abstraction to have in mind for this course
- A special register called *program counter*(PC) or *instruction pointer*(IP) is used to keep track of the current instruction being executed
 - In x64 we call this register *RIP*
 - The ISA does not allow us to access this register directly, but indirectly using jump instructions

Program flow cont.

Assembly

- Jump instructions break the sequential flow of programs
- In higher-level languages like C or Rust:
 - *function calls*
 - *loops*
 - *conditional statements*
- Translates into jump instructions
- In assembly we can define places to jump to using 'labels' (see next slide)

- We have two types of jump instructions: conditional or unconditional
- Unconditional jumps performs a jump to somewhere without any rules attached
- Conditional jumps on the other hand
 - Will check a so called *flags*-register to determine if they should jump or continue to the sequential next instruction
 - There exist instructions to set these flags, and we will use the *cmp*-family of instructions
- This is enough to create *if*, *loop*, etc.

Jump example

Assembly

```
1  movl $4, %eax    # EAX = 4
2  movl $10, %ebx   # EBX = 10
3  cmpl %eax, %ebx  # Compare operands
4  jae jump_here    # Jump if EBX >= EAX
5
6  #Code below the jump statement will
7  #not be executed
8
9
10 jump_here:
11     #Code here will be executed
```

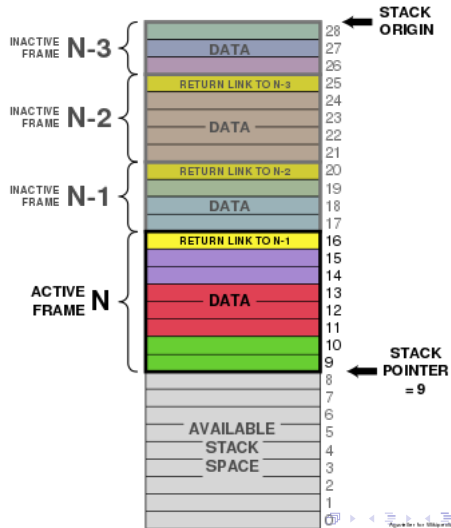
Instruction	Jump if	Meaning
JE	Equal	A == B
JNE	Not equal	A != B
JB	Below (unsigned)	A < B
JA	Above (unsigned)	A > B
JBE	Below or equal (US)	A <= B
JAЕ	Above or equal (US)	A >= B
JL	Less (signed)	A < B
JG	Greater (signed)	A > B
JLE	Less or equal (S)	A <= B
JGE	Greater or equal (S)	A >= B

Stack

Stack

- The hardware stack is memory that is accessed in a last-in-first-out manner
- In previous courses you have perhaps implemented software stacks, and it's exactly the same concept
- In x86 the stack 'grows' **downwards** in memory, from a high address to a low address

source: stack based memory



- In order to keep track of the stack we use two registers *Stack pointer*(RSP), and *Base pointer*(RBP)
- These two registers allows us to easily store
 - Function variables (local)
 - temporary data
 - functions arguments
- That we would otherwise have to manually request memory for

Stack cont.

Stack

- The stack pointer(RSP) points to the 'top' of the stack, where we put a value last.
- Data below the stack pointer is considered garbage
- The base pointer(RBP) points to the 'bottom' of a so called *stack-frame*.
- A stack frame is a portion of the total stack, and a local area for a particular function.

- The x86-ISA defines two operations that makes working with the stack easier:
 - *push*: Adds something to the stack
 - *pop*: Removes something from the stack
- The stack is currently 16-bit aligned, so there is no such thing as pushb/popb
- A good rule of thumb is to always keep symmetry between your pushes and pops

- The stack operations affect the stack pointer(RSP), since it always points to the top of the stack
 - push will implicitly do: `sub <size> rsp, mov <val> (%rsp)`
 - pop will implicitly do: `add <size> rsp`
- You should not have to deal with endiannes, but worth noting:
 - The x86-family is *little-endian*
- If the register **EAX** has the value 0x012334567, and we want to move it into memory address 0xABCD0000
 - How will the four bytes that is the value of EAX be located in relation to the address?

Endianness

Stack

Value: 0x01234567

Big-endian

Address	Byte
0xABCD0000	0x01
0xABCD0001	0x23
0xABCD0002	0x45
0xABCD0003	0x67

Little-endian

Address	Byte
0xABCD0000	0x67
0xABCD0001	0x45
0xABCD0002	0x23
0xABCD0003	0x01

Adding/subtracting on the stack pointer

Stack

- The stack pointer is constantly moving around, creating stack frames, pushing/popping values
- The base pointer is usually static after a new *stack frame* has been set up
- To create a new stack frame:
 - Store the old value of the *base pointer*
 - Copy the value of the *stack pointer* into the base pointer
- Voila! A new stack frame
- To clean up and return to the previous stack frame:
 - Pop up the stack / add to the stack pointer
 - Pop the stored base pointer value

Calling convention

Functions

- There exist different calling conventions. We will use **System V AMD64 ABI**
- This sets some rules for us when we call and create functions
- We divide our registers into two groups:
 - *volatile*(caller-save/scratch)
 - *non-volatile*(callee-saved)

Calling convention cont.

Functions

- The caller-saved registers are:
 - RAX, RCX, RDX, R8, R9, R10, R11
- The callee-saved registers are:
 - RBX, RBP, RDI, RSI, RSP, R12, R13, R14, R15

Calling convention cont.

Functions

- The calling convention also specifies where return values should be put
- **RAX** is used for integer values up to 64-bit, for 128-bit integer values, RAX is combined with **RDX**
- There are more types of values
- For more details see: https://www.uclibc.org/docs/psABI-x86_64.pdf

Calling functions

Functions

- To call a function(label) in assembly we can use the **call** instruction
- This will save/push the *instruction pointer* onto the stack
 - This means we can return to wherever we came from after a function is done
 - The function to 'undo' a call is called **ret**
- This is why having symmetry and control over the stack pointer is so important, if we return and pop the wrong value for the instruction pointer, we will end up in weird places

Jumps

examples

```
1  movl $1, %ebx # b = 1;
2  cmpl $5, %eax # compare a with 5...
3  jne after      # ..and if a != 5, jump to .after
4  # Else, if a == 5...
5  addl $2, %ebx # b += 2
6  .after:
7  incl %ebx      # b++
```

```
1  int b = 1;
2  if (a == 5)
3  {
4      b += 2;
5  }
6  b++;
```

```
1  movl $0, %eax # int a = 0;
2  movl $0, %ecx # int i = 0;
3▼ loop:
4      cmpl $10, %ecx # if ECX...
5      jge end; # ...is greater or equal 10, jump to end
6      addl %ecx, %eax # a += i;
7      incl %ecx # i++;
8      jmp loop # goto loop;
9▼ end:
10     incl %eax # a++;
```

```
1  int i, a = 0;
2  for (i = 0; i < 10; i++)
3      a += i;
4
5  a++;
```

- Moore's law
- Calling conventions
- GAS assembly
- x64 cheat sheet
- System V ABI
- x64 registers
- Compiler explorer
- Agner Fog's research (very,very good)

Questions?